UNIT — SEARCH ENGINES & THE WEB

Search Engines & Web Browsers

How web-scale Information Retrieval actually works, end to end

Information Retrieval Systems • Final Year B.E. IT, Sem VII • Mumbai University

Reference: Introduction to Information Retrieval — Manning, Raghavan & Schütze (Ch. 19–21)

ROADMAP FOR TODAY

What We'll Cover

The Big Picture

What a search engine is, and how it differs from the database-style retrieval you already know.

Anatomy of a Search Engine

The offline pipeline (crawl → index) and the online pipeline (query → rank).

Crawling, Indexing & Ranking

How pages are gathered, stored for fast lookup, and ordered by relevance — including PageRank.

Where the Browser Fits In

Why the browser is the client, not part of the retrieval intelligence — and the full journey from keystroke to results.

DEFINITION

What Is a Search Engine?

A search engine is a live, working implementation of everything we've studied this semester:

an Information Retrieval system operating at the scale of the entire World Wide Web.

It takes a free-text query from a user, searches billions of unstructured documents it does not own or control, and returns a ranked list of the most likely-relevant pages — in well under a second.

Everything we've studied is inside it

- Objective 1–5 (need, precision, recall, ranking, scale)
- Inverted indexing, at web scale
- Ranked retrieval (not Boolean, not SQL)
- Precision/recall trade-offs, every single query

WHY WEB SEARCH IS UNIQUELY HARD

The Web Is a Different Beast

> *"The Web is unprecedented in many ways: unprecedented in scale, unprecedented in the almost-complete lack of coordination in its creation, and unprecedented in the diversity of backgrounds and motives of its participants."*
>
> — *Manning, Raghavan & Schütze, Ch. 19*

Massive Scale

Billions of documents, spread across millions of independently-run servers — not one tidy collection.

No Coordination

Nobody planned the Web's structure. Anyone can publish anything, in any format, at any time.

Adversarial

Ranking = visibility = money, so an entire industry exists to manipulate search results (SEO, spam).

Hyperlinked

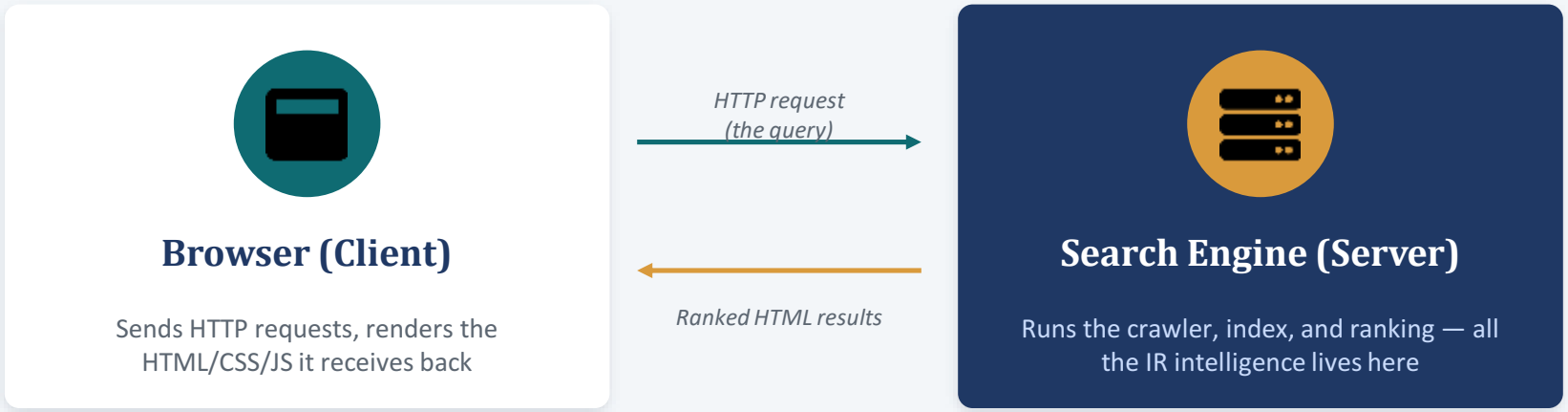
Pages link to each other — a structure traditional document collections (like Shakespeare's works) never had.

THE FOUNDATION

Client–Server: Browser Meets Search Engine

"...a simple, open client–server design: the server communicates with the client via HTTP — a protocol that is lightweight and simple."

— Manning, Raghavan & Schütze, Ch. 19

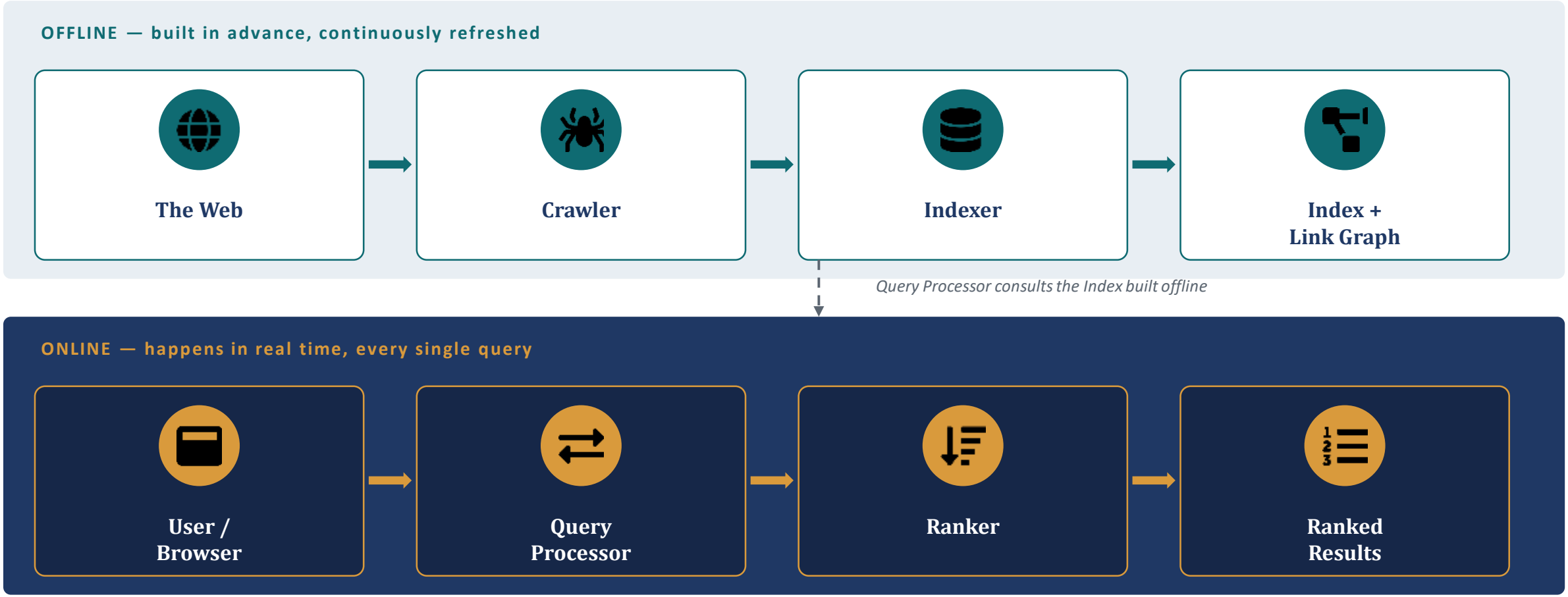


Key idea

The browser is dumb by design. It never crawls, indexes, or ranks anything — it only displays what the server sends back.

THE BIG PICTURE

Anatomy of a Search Engine



COMPONENT 1 OF 4

The Web Crawler (a.k.a. Spider)

“Web crawling is the process by which we gather pages from the Web, in order to index them and support a search engine.”

— Manning et al., Ch. 20

Goal: gather as many useful pages as possible — together with the link structure connecting them.

How it actually works — the basic loop

- Start with a seed set of known URLs.
- Pick a URL, fetch that web page over HTTP.
- Parse the page: pull out its text AND its links.
- Send the text to the indexer.
- Add newly-found links to the URL frontier — the queue of pages still to be fetched.
- Repeat, forever — this is literally traversing the entire web graph, one page at a time.

Two Things a Crawler MUST Get Right

Robustness

Spider traps: some sites (accidentally or maliciously) generate an infinite number of pages — a calendar with a “next day” link forever, for instance.
A naive crawler can get stuck fetching one domain forever, wasting all of its time and never reaching the rest of the Web.

A production crawler must detect and escape these traps.

Politeness

robots.txt: a file at the root of a site telling crawlers which parts they may NOT visit. The crawler must respect it.
Rate limits: only one connection open per host at a time, with a real gap (seconds) between requests to the same server — never hammer a single site with requests.

Without this, crawlers would function like a denial-of-service attack on every site they visit.

Inside One Crawl Cycle

1

Fetch

Download the page at the URL via HTTP

2

Parse

Extract the text AND the outgoing links

3

Dedup

“Content seen?” — skip if this page already exists elsewhere

4

Filter

Apply robots.txt + URL normalization

5

Frontier

Add surviving new links to the queue

Extracted TEXT goes to the Indexer. Extracted LINKS (with anchor text) go on to feed Ranking later.

A HIDDEN BOTTLENECK

Before You Can Even Fetch a Page...

DNS Resolution

A URL like www.wikipedia.org isn't an address a computer can connect to directly.

It first has to be translated into an IP address, e.g. 207.142.131.248 — this lookup is called DNS resolution.

This lookup can take seconds, and standard library lookups block the entire thread while waiting.

Why this matters at scale

Fetching a billion pages in a month-long crawl needs several hundred fetches EVERY SECOND.

If every fetch had to wait for a blocking DNS lookup, the whole crawler would grind to a crawl (pun intended).

Fix: crawlers build their own asynchronous DNS resolver, plus aggressive caching of recent lookups, so hundreds of fetches can be in flight at once.

COMPONENT 2 OF 4

The Indexer — At Web Scale

Recap from Unit 1: an inverted index maps every term to the documents it appears in, so a query becomes a lookup, not a scan through billions of pages.

The problem

One machine cannot hold an index for the entire Web — billions of pages, hundreds of billions of postings. The index itself must be split across many machines.

Option A: Term partitioning

Split the dictionary itself across nodes — each node owns a subset of terms. Sounds elegant, but multi-word queries require shipping large postings lists between nodes to merge — expensive in practice.

Option B: Document partitioning

Split the documents across nodes — each node holds a full mini-index for its subset of documents. Query is broadcast to every node; top results are merged. This is the more common, better-performing choice in practice.

COMPONENT 3 OF 4

Link Analysis — Using the Web's Own Structure

The core idea

A hyperlink from page A to page B is a vote: the author of A thought B was worth linking to. Millions of such “votes” across the Web are a powerful relevance signal that plain keyword matching cannot see.

Anchor text: the clickable text of a link also describes the page it points to — used as an extra relevance clue for the target page.

PageRank — the “random surfer” model

- Imagine a surfer who starts on some random page and just keeps clicking random links, forever.
- Pages visited more often, over the long run, get a higher score — because many other pages chose to link there.
- A page linked to by many important pages ranks higher — even with the exact same keywords as a page nobody links to.

COMPONENT 4 OF 4

Query Processor & Ranking

Two very different kinds of evidence get combined into ONE final score for every candidate page:

Content signals

How often and how prominently the query terms appear in the page (term weighting / tf-idf — a later topic).

+

Link signals

PageRank and anchor text — how the Web's own link structure votes on a page's importance.

=

Final ranked list

Sorted by combined score, best guess first.

This is the Query Processor from the master diagram — it consults the offline Index + Link Graph, scores every candidate, and hands a ranked list straight back to the browser as HTML. All in real time, on every single query.

CLEARING UP A COMMON CONFUSION

So... What Does the Browser Actually Do?

The Browser DOES:

- Turn your typed query into an HTTP request
- Send that request to the search engine's server
- Receive the ranked HTML page back
- Render it visually — text, layout, images, links you can click

The Browser NEVER Does:

- Crawl the Web
- Build or store an inverted index
- Compute relevance scores or PageRank
- Decide the order of the results

PUTTING IT ALL TOGETHER

From Keystroke to Results: The Full Journey

1. You type a query and hit Enter in your browser.
2. Browser resolves the search engine's domain via DNS, opens an HTTP connection.
3. Your query is sent as an HTTP request to the search engine's server.
4. Query Processor consults the pre-built Index + Link Graph.
5. Ranker scores every candidate page (content signals + link signals).
6. Server sends back a ranked HTML page of results.
7. Browser renders that HTML — this is the results page you see and click.

Colour key

- **Browser side**

Steps 1–3, 7

- **Server / online**

Steps 4–5

- **Result delivery**

Step 6

IT'S MESSIER IN PRACTICE

Real-World Challenges at Web Scale

Spam & SEO

"...not being fooled by site providers manipulating page content in an attempt to boost their search engine rankings, given the commercial importance of the web." — Manning et al., Ch. 1. Ranking well = traffic = money, so manipulation is constant and adversarial.

Near-Duplicate Pages

Mirrors, syndicated articles, and boilerplate content mean many pages are near-identical. Search engines use shingling to detect and avoid ranking redundant copies.

Crawl Traps & Politeness

Infinite calendar pages, rate limits, and robots.txt all have to be handled correctly, at billions-of-pages scale, without ever taking down a small site by accident.

QUICK REVISION

Key Terms — One-Glance Recap

Crawler / Spider

Program that fetches web pages, extracts text + links, and feeds the indexer.

URL Frontier

The queue of URLs still waiting to be fetched by the crawler.

robots.txt

A file telling crawlers which parts of a site they may not visit.

Inverted Index

Term → document lookup table; turns a search into a fast lookup, not a scan.

Document Partitioning

Splitting the web-scale index across machines by document, not by term.

Anchor Text

The clickable text of a link; used as a relevance clue for the linked page.

PageRank

Link-based importance score, based on a random surfer's long-run visit frequency.

Query Processor

Real-time component that scores and ranks candidates for a live query.

Shingling

Technique for detecting near-duplicate web pages efficiently.

WRAPPING UP

One Sentence to Remember

A search engine is just an Information Retrieval system, built for the entire Web — crawl offline, index offline, then match, rank, and respond online, in real time.

Exam pointers for this topic

- Draw the offline (crawl → index) vs. online (query → rank) split from memory.
- Explain crawler robustness (spider traps) and politeness (robots.txt) with examples.
- State clearly why the browser is a client, never part of the retrieval pipeline.
- Explain how content signals and link signals (PageRank) combine into one ranking score.

Reference: Introduction to Information Retrieval — Manning, Raghavan & Schütze, Chapters 19–21